



SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

ODJEL ZA MATEMATIKU

Sveučilišni preddiplomski studij Matematika i računarstvo

UI sustav za dinamičko povećanje rezolucije i generiranje umjetnih okvira u interaktivnoj grafici

ZAVRŠNI RAD

Mentor:

**izv. prof. dr. sc. Domagoj Še-
verdija**

Kandidati:

**Ante Galić
Ivo Janković**

Osijek, 2026.

Sadržaj

1	Uvod	1
2	Teorijska podloga	3
2.1	Uzorkovanje i aliasing	3
2.2	Temporalna akumulacija	3
2.3	Procjena gibanja i optical flow	4
2.4	Reprojekcija i disokluzija	5
3	Pregled postojećih rješenja	7
3.1	NVIDIA DLSS	7
3.2	AMD FSR	7
3.3	Intel XeSS	8
3.4	Apple, Sony, Arm, Qualcomm	8
3.5	Istraživački modeli interpolacije okvira (VFI)	8
3.6	Presjek i veza s ovim radom	8
4	Arhitektura vlastitog sustava	11
4.1	Odluke i njihovo obrazloženje	11
4.2	Cjevovod	11
4.3	Ground truth: ključna prednost postavke	12
4.4	Razvojni hardver	12
5	Implementacija upscalera	13
5.1	Prostorni baseline (nearest, bilinear, bicubic, FSR1)	13
5.2	Minimalni temporalni upscaler (TAAU)	14
5.3	Puni upscaler (FSR3-stil)	16
6	Implementacija procjene gibanja	19
6.1	Piramidalni block matching	19
6.2	Referenca i točnost	20
6.3	Ablacije	20
6.4	Detekcija promjene scene	21
6.5	Cijena i granice	21
7	Implementacija generiranja okvira	23
7.1	Izgradnja međuokvira	23
7.2	Rezultati	24

7.3	Prikaz: frame pacing, latencija, UI	24
8	Naučena komponenta	27
8.1	Što mreža radi	27
8.2	Arhitektura	27
8.3	Skup podataka i učenje	28
8.4	Rezultati	28
8.5	Cijena	28
8.6	Demonstracija	29
9	Metodologija mjerenja	31
9.1	Ground truth	31
9.2	Skriptirana kamera i determinizam	31
9.3	Mjerenje performansi	32
9.4	Mjerenje kvalitete: PSNR i SSIM na GPU-u	32
9.5	Alat za regeneraciju tablica	32
10	Rezultati	33
10.1	Provjera na drugom hardveru	33
10.2	Sinteza po modulu	34
10.3	Latencija	34
10.4	Provjera na dva GPU-a proizvođača	35
10.5	Artefakti	35
11	Rasprava i ograničenja	37
11.1	Odstupanja od izvornog plana	37
11.2	Ograničenja mjerne postavke	37
12	Zaključak i mogući nastavak	39
12.1	Doprinosi	39
12.2	Mogući nastavak	39
	Literatura	41
	Sažetak	43
	Summary	45
	Životopis	47

1 | Uvod

Razlučivost i frekvencija osvježavanja zaslona rastu brže od rasterizacijske snage grafičkih procesora. Posljedica je da igre i interaktivne 3D aplikacije sve češće ne renderiraju svaki piksel svakog prikazanog okvira izravno, nego rekonstruiraju veći dio slike iz manjeg broja stvarno izračunatih uzoraka — kroz prostor (*upscaling*) i kroz vrijeme (*generiranje interpoliranih okvira*). AMD-ov FidelityFX Super Resolution 3 (FSR3) jedan je od nekoliko komercijalnih sustava koji te dvije tehnike kombinira u jedan cjevovod; ovaj rad ne koristi FSR3 kao gotovu biblioteku, nego iz opisa njegovih tehnika (dostupnih kao javna dokumentacija, ne kao izvorni kod) iznova implementira ključne dijelove: temporalni upscaler, procjenu gibanja iz slike (*optical flow*) i generiranje međuokvira, u vlastitom cjevovodu pisanom od nule u C++20 i OpenGL 4.6 compute shaderima.

Ključna postavka koja cijeli rad čini mjerljivim, a ne dojmljivim, je vlastiti renderer testne scene: kako se scena i kamera u cijelosti kontroliraju, moguće je renderirati i *ground truth* — puni native okvir kao referencu za upscaler, i pravi međuokvir na trenutku $t - 0,5\Delta t$ kao referencu za interpolaciju. Kvaliteta rezultata je time PSNR/SSIM brojka naspram poznate istine, ne subjektivan dojam iz igranja.

Rad je podijeljen u dvanaest poglavlja. Poglavlje 2 postavlja teorijsku podlogu (uzorkovanje i aliasing, temporalna akumulacija, procjena gibanja, reprojekcija i disokluzija). Poglavlje 3 daje pregled postojećih komercijalnih i istraživačkih rješenja i smješta ovaj rad među njih. Poglavlje 4 opisuje arhitekturu vlastitog sustava. Poglavlja 5–7 opisuju implementaciju upscalera, procjene gibanja i generiranja okvira, pass po pass. Poglavlje 8 opisuje naučenu komponentu — malu mrežu koja uči birati između kandidata koje klasični dio cjevovoda već proizvodi. Poglavlje 9 opisuje metodologiju mjerenja, s naglaskom na ground-truth referencu. Poglavlje 10 donosi rezultate: kvalitetu, performanse, latenciju, ablacije i provjeru na dva različita GPU-a. Poglavlje 11 raspravlja ograničenja i odstupanja od izvornog plana rada, a Poglavlje 12 zaključuje i predlaže moguće nastavke.

2 | Teorijska podloga

2.1 Uzorkovanje i aliasing

Rasterizacija procjenjuje integral svjetlosti koji piksel prima jednim ili malim brojem uzoraka po pikselu. Prema teoremu uzorkovanja, signal koji sadrži frekvencije iznad polovice frekvencije uzorkovanja (Nyquistove frekvencije) ne može se iz uzoraka jednoznačno rekonstruirati — visoke frekvencije se *preklope* (alias) u niske i pojave se kao vidljivi artefakti: nazubljeni rubovi, treperenje tankih detalja, moiré uzorci na teksturama visoke frekvencije. Rješenje nije bolji filter nakon uzorkovanja, nego **više uzoraka prije** njega — supersampling, ili, ekonomičnije, uzorci raspoređeni tako da se njihov prosjek kroz vrijeme približi gušćem uzorkovanju.

Ovo je razlog zašto renderer u ovom radu, kad treba *referencu* za mjerenje kvalitete, ne koristi jednostruki render u prikaznoj razlučivosti, nego 2×2 supersampling usrednjen u linearnom prostoru boje prije tone mappinga (Poglavlje 9): bodovanje naspram aliasirane referencije bi sustavno nagrađivalo zamućenje, jer zamućeni filter “pogodi” aliasiranu referencu bliže u srednjekvadratnoj pogrešci nego filter koji ispravno rekonstruira rub.

2.2 Temporalna akumulacija

Ako se kamera svaki okvir pomakne za pod-pikselni razmak (*jitter*) i rezultati se akumuliraju kroz vrijeme s odgovarajućim ponderiranjem, niz okvira zajedno djeluje kao gušće uzorkovanje jednog okvira — temporalni antialiasing (TAA). Karis [2] formalizira ovaj pristup za igre u stvarnom vremenu: svaki okvir doprinosi jedan novi uzorak po pikselu na novoj sub-pikselnoj fazi, a povijest (prethodni akumulirani rezultat) donosi sve prijašnje. Niz faza uzorkovanja obično se bira iz Haltonove sekvence (baza 2, 3), jer nizak-discrepancy niz jednoliko popunjava jedinični kvadrat brže od slučajnog uzorkovanja.

Cijena je da akumulacija zahtijeva **reprojekciju**: povijest je snimljena za prošlu poziciju kamere (i prošle položaje pomičnih objekata), pa se prije miješanja s trenutnim okvirom mora preslikati na trenutnu geometriju pomoću vektora gibanja. Yang i sur. [3] u svom pregledu tehnika temporalnog antialiasinga sustavno klasificiraju izvore pogreške ovog koraka: pogrešni ili nepostojeći vektori gibanja (sjene, refleksije, promjene sjenčanja nisu opisane geometrijskim gibanjem), **disokluzija** (površina koja u prošlom okviru nije bila vidljiva nema valjanu povijest) i **ghosting** (povijest koja se pogrešno prihvati kao valjana kad zapravo ne

pripada trenutnoj površini). Sve tri kategorije izravno odgovaraju mehanizmima opisanima u Poglavlju 5 (depth clip za disokluziju, color-box clamp za ghosting, dilatacija vektora za rubove objekata).

Temporalna akumulacija je, po konstrukciji, niskopropusni filter u vremenu: prigušuje visoke frekvencije koje se ne poklapaju savršeno između okvira. To je poanta kad je riječ o aliasingu, ali postaje mana kad se scena stvarno mijenja brže nego što akumulacija stigne pratiti — analitički izvod tog efekta (formula (5.1)) jedan je od glavnih nalaza Poglavlja 5.

2.3 Procjena gibanja i optical flow

Vektor gibanja dobiven iz rasterizacije (razlika projicirane pozicije istog vrha geometrije između dva okvira) postoji samo za ono što se doista crta trokutima, s poznatom transformacijom. Optical flow rješava opći problem: procijeniti polje gibanja **izravno iz dvije slike**, bez pristupa geometriji koja ih je proizvela.

Klasična formulacija (Fleet i Weiss [1]) polazi od pretpostavke **konstantnosti svjetline**: sjaj točke na površini se ne mijenja između okvira, samo se pomiče,

$$I(x, y, t) = I(x + u \delta t, y + v \delta t, t + \delta t), \quad (2.1)$$

gdje je (u, v) traženi vektor gibanja. Linearizacijom (2.1) Taylorovim redom dobiva se jednadžba optičkog toka $I_x u + I_y v + I_t = 0$ — jedna linearna jednadžba s dvije nepoznanice po pikselu, dakle problem je **nedovoljno određen** (*ill-posed*). Dvije klasične posljedice te nedovoljne određenosti:

- **Aperture problem.** Unutar homogene ravne plohe ili duž ruba koji je lokalno pravac, gradijent I_x, I_y ne određuje komponentu gibanja duž tog ruba — sve vrijednosti te komponente daju isti lokalni odgovor. Rješava se dodatnim pretpostavkama (glatkoća polja, veći prozor za promatranje) ili prihvatanjem da procjena u takvim područjima nasljeđuje grublju/susjednu procjenu.
- **Loša postavljenost problema** u područjima niskog kontrasta ili periodične strukture — više različitih vektora gibanja daje jednako dobro podudaranje (isti minimum kriterija), pa je izbor osjetljiv na šum.

Ove dvije pojave nisu apstraktne u ovom radu: izravno objašnjavaju zašto implementirani procjenitelj (Poglavlje 6) treba kaznu udaljenosti od predikcije (rješava lošu postavljenost) i zašto sustavno griješi na periodičnim teksturama i ravnim plohama (aperture problem), a mjereno je koliko griješi, ne samo zašto.

Praktične implementacije za grafiku u stvarnom vremenu, uključujući `FfxOpticalFlow` koji je uzor implementaciji u ovom radu, ne rješavaju linearizirani sustav diferencijalno (Lucas–Kanade), nego **diskretnom pretragom** (block matching): za blok piksela luminancije traži se pomak koji minimizira kriterij razlike (npr. sumu apsolutnih razlika) unutar prozora pretrage, u piramidi razlučivosti *coarse-to-fine* da bi se pokrili i veliki pomaci. Ovo je aproksimacija koja žrtvuje sub-pikselnu preciznost i teorijsku elegantnost gradijentnih metoda za robusnost na velike, brze pomake tipične za igre — kompromis koji je u ovom radu izmjeren, ne samo naveden (Poglavlje 6).

2.4 Reprojekcija i disokluzija

Reprojekcija — dohvaćanje vrijednosti iz prošlog okvira na poziciji koju vektor gibanja pokazuje — pretpostavlja da površina koja je tamo bila i dalje postoji i vidljiva je. Dvije pretpostavke mogu propasti: povijest s te pozicije nikad nije zapisana za *ovu* površinu (nova površina se otkriva — disokluzija), ili je zapisana za drugu, sada prekrivenu površinu (obrnuta disokluzija). Standardan test je **provjera dubine**: usporedba dubine koju je površina imala prošli okvir (poznata ili rekonstruirana) s dubinom koju prošli dubinski spremnik ima na reprojiciranoj poziciji — neslaganje iznad praga znači da povijest ne pripada ovoj površini i mora se odbaciti ili umanjiti njezin utjecaj. Ovo je princip iza *depth clip* mehanizma opisanog u Poglavlju 5 i iza dviju maski disokluzije u generiranju međuokvira (Poglavlje 7).

3 | Pregled postojećih rješenja

Ovo poglavlje smješta rad među postojeća komercijalna i istraživačka rješenja za upscaling i generiranje okvira. Detaljnije bilješke s izvorima po tvrdnji (URL-ovi, razlikovanje potvrđenih i nepotvrđenih navoda) čuvaju se odvojeno od ovog rada; ovdje je sažetak i, na kraju poglavlja, izravna veza sa svakim implementiranim modulom.

3.1 NVIDIA DLSS

DLSS 2 (2020.) [8] uvodi naučenu temporalnu rekonstrukciju: konvolucijski auto-encoder na Tensor jedinicama prima boju niske razlučivosti, igrine vektore gibanja i prethodni izlaz visoke razlučivosti, treniran naspram 16K offline-renderirane referentne slike. DLSS 3 (2022.) dodaje generiranje okvira kao zaseban dio: ulazi su dva okvira, hardverski optical flow (Ada Optical Flow Accelerator) i igrini podaci; zahtijeva RTX 40. DLSS 4 (2025.) mijenja mrežu za super-rezoluciju s CNN-a na transformer i seli generiranje okvira s hardverskog optical flowa na naučeni model. DLSS 4.5 (2026.) dodaje drugu generaciju transformera i FP8 na RTX 40/50.

Reflex 2 Frame Warp [11] je iznimka među NVIDIA-inim rješenjima: kasna reproprojekcija (ekstrapolacija, ne interpolacija) koja iz zadnjeg unosa i zadnjeg renderiranog okvira izračuna novu kameru i warpa okvir neposredno prije prikaza.

3.2 AMD FSR

FSR 1 je čisto prostorni, klasičan pristup (EASU + RCAS) — ista dva mehanizma implementirana u ovom radu (Poglavlje 5). FSR 2 je temporalan i klasičan. **FSR 3/3.1 [7] je klasičan i u dijelu za generiranje okvira:** optical flow (temelj AMD Fluid Motion Frames) radi po blokovima 8×8 bez preklapanja, piramida s SAD pretragom — struktura koju M6 ovog rada slijedi, s gušćim poljem (vektor po pikselu, ne po bloku). FSR 4 (2025.) je naučeni upscaler na FP8 matričnim jedinicama RDNA 4; FSR “Redstone” (2025.) dodaje naučeno generiranje okvira koje se mijenja s klasičnom reproprojekcijom vektora. Za Polaris (RX 400/500, hardver za koji je ovaj rad izvorno planiran) AMD nema izjavu — vrijede samo klasični FSR 1/2/3.

3.3 Intel XeSS

XeSS 2 (2024.) [9] je komercijalno rješenje najbliže M9 ovog rada: tri stupnja — reprojekcija igrinim vektorima, naučeni model za reprojekciju optičkim tokom, i **drugi naučeni model s funkcijom mješavine koji bira između oba kandidata**. To je gotovo doslovno arhitektura M9-a (mreža ne crta piksele, bira/miješa kandidate), samo s naučenim, ne klasičnim, optičkim tokom kao jednim od kandidata, i na hardveru s XMJ jedinicama.

3.4 Apple, Sony, Arm, Qualcomm

Apple MetalFX ima temporalni naučeni upscaler (od 2022.) i interpolator okvira (od 2025.). Sony PSSR (PlayStation 5 Pro, 2024.) je naučeni upscaler na namjenskom ML hardveru; nadograđeni PSSR (2026.) dolazi iz zajedničkog razvoja s AMD-om i koristi isti model kao FSR Upscaling, treniran na drugim podacima, u INT8. **Arm Neural Super Sampling** [10] (najavljen 2025.) je konceptualno najbliži analog M9 pristupa u cijelom pregledu: UNet predviđa filterske jezgre i temporalne koeficijente, a klasični akumulator ih primjenjuje — mreža opet ne crta piksele nego parametrizira klasični dio. Qualcommov Snapdragon GSR je klasičan (bez ML-a).

3.5 Istraživački modeli interpolacije okvira (VFI)

RIFE, FILM, IFRNet i EMA-VFI (2022.–2023.) rade isključivo iz RGB parova okvira, bez igrinih vektora, dubine ili odvajanja korisničkog sučelja — moraju procijeniti cijelo gibanje samo iz piksela. Cijena je desetci do stotine milisekundi na vrhunskim GPU-ima (RIFE: 16 ms na 480p na TITAN X Pascal), naspram budžeta od ≈ 8 ms za cijeli skok 60→120 fps. RIFE ima 9,8 milijuna parametara — 1361× više od mreže u ovom radu (7199 parametara). Trenirani su na prirodnom videu, ne na aliasiranim, jitteriranim renderima s korisničkim sučeljem, i nemaju integraciju s pacingom ili Reflex-stilskim smanjenjem latencije — zato se ne koriste izravno u igrama.

3.6 Presjek i veza s ovim radom

Zajednički obrazac komercijalnih rješenja: inferencija je gusto matrično množenje u 8-bitnim formatima (FP8 na RTX 40/50 i RDNA 4, INT8 na RDNA 3 portu i na PS5 Pro, XMJ/DP4a na Intelu). Bez namjenske 8-bitne podrške isti model košta znatno više — točno razlog zašto FSR 4 na RDNA 3 mora biti prekvantiziran u INT8, i točno razlog zašto poglavlje 8 ovog rada mjeri granicu male naučene mreže **bez** takvog hardvera. Svi komercijalni sustavi za generiranje okvira **interpoliraju** (drže zadnji stvarni okvir, pa idu uz Reflex/Anti-Lag 2/XeLL za latenciju) — isto radi i ovaj rad (Poglavlje 7), uz izmjerenih +8–16 ms latencije. Svi odvajaju korisničko sučelje od scene — isto radi M8 ovog rada.

Rješenje	Proizvođač	Vrsta	Klasično / naučeno
DLSS 3/4 FG	NVIDIA	FG	naučena sinteza, (bivše) HW optical flow
FSR 3/3.1	AMD	SR+FG	klasično; block-SAD optical flow 8×8
FSR Redstone FG	AMD	FG	naučeno gibanje+izgled + reprojekcija
XeSS 2 FG	Intel	FG	reprojekcija + naučeni tok + naučena mješavina
Arm NSS	Arm	SR	naučen UNet, jezgre za klasični akumulator
RIFE/FILM/...	istraživanje	VFI	naučen tok, samo iz RGB parova
ovaj rad	—	SR+FG	klasično (M3–M8) + naučena mješavina (M9)

Tablica 3.1: Sažetak usporedbe; puna tablica sa svim izvorima u pratećoj dokumentaciji rada.

4 | Arhitektura vlastitog sustava

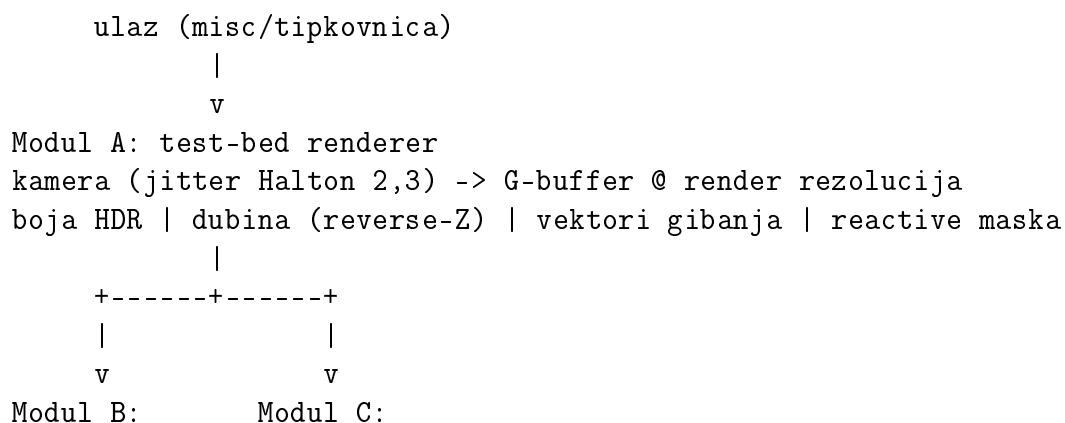
4.1 Odluke i njihovo obrazloženje

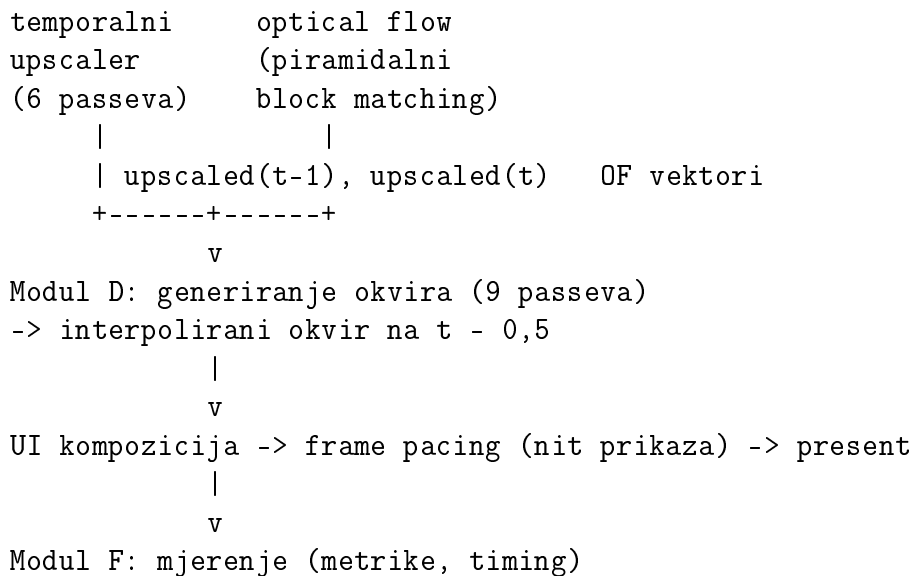
Odluka	Izbor
Jezik / API	C++20 + OpenGL 4.6 compute shaderi (GLSL 460)
Scena	vlastiti mali renderer + glTF (Sponza, Damaged Helmet)
Referentne implementacije	čitaju se kao opis algoritma, ne kopiraju
Naučeni dio	mali modul + poglavlje analize, ne zamjena jezgre

Tablica 4.1: Ključne odluke plana rada.

OpenGL 4.6 compute shaderi izabrani su nad Vulkanom jer sadrže sve potrebne značajke (imageAtomicMax nad r32ui formatom, nužno za scatter polja vektora u generiranju okvira; GPU timestamp queries; storage buffere) uz znatno manje infrastrukturnog koda — vrijeme rada ide u algoritam, ne u rukovanje sinkronizacijom i memorijskim barijerama koje Vulkan traži eksplicitno. Vlastiti mali renderer (ne gotov engine) je nužan, ne stilski izbor: bez izravne kontrole nad jitterom kamere, dubinskim spremnikom i vektorima gibanja temporalni upscaling se ne može ni implementirati ni mjeriti — gotov engine bi te podatke sakrio iza vlastite abstrakcije. Referentna FSR3 dokumentacija (javno dostupni API-neutralni opisi tehnika) čita se kao pseudokod; implementacija u ovom radu je vlastita, iz opisa, a ne prijepis izvornog koda FidelityFX SDK-a.

4.2 Cjevovod





Modul B (Poglavlje 5) i Modul C (Poglavlje 6) rade paralelno na istim ulazima iz Modula A i hrane Modul D (Poglavlje 7), koji rezultat generiranja predaje redu prikaza. Modul E, naučena komponenta (Poglavlje 8), djeluje unutar Modula D, kao zamjena za ručno napisano pravilo miješanja kandidata u prolazima 10–12. Modul F (Poglavlje 9) nije zaseban dio cjevovoda nego instrumentacija koja prati sve ostale module.

4.3 Ground truth: ključna prednost postavke

Renderer u ovom radu, jer je vlastiti, može renderirati dva oblika *ground trutha* koje gotov engine ne izlaže: **puni native okvir** u prikaznoj razlučivosti bez jittera (referenca za upscaler) i **pravi okvir na trenutku** $t - 0,5\Delta t$, dobiven pomicanjem scene i kamere na taj trenutak i renderiranjem bez akumulirane povijesti (referenca za generiranje okvira). Oba se snimaju supersamplirano (2×2) i usrednjena u linearnom prostoru boje prije tone mappinga (obrazloženje u Poglavlju 2, odjeljak o uzorkovanju).

Ova postavka je razlog zašto se kvaliteta u ovom radu izvještava kao PSNR/SSIM brojka naspram poznate istine, umjesto kao subjektivan dojam — i razlog zašto su ablacije u poglavljima 5–8 mjerljive, a ne samo opisane.

4.4 Razvojni hardver

Rad je razvijan na GPU-u AMD Radeon RX 580 (Polaris, Mesa upravljački program), izabranom jer nema namjenske matrične/tenzorske jedinice — okolnost koja izravno motivira mjerenje granice naučene komponente u Poglavlju 8. Kompletно mjerenje ponovljeno je i na dva novija GPU-a (NVIDIA RTX 5070 i AMD RX 7800 XT), s rezultatima u odjeljku 10.1, kao provjera da zaključci ne ovise o jednom uređaju.

5 | Implementacija upscalera

Upscaler se u ovom radu razvijao u tri koraka, svaki mjerljiv zasebno: prostorni baseline (bez ikakve vremenske komponente), minimalni temporalni upscaler i, naposljetku, puni upscaler u stilu AMD FSR3. Svaki sljedeći korak mora nadmašiti prethodni na istoj sceni i istoj metrici, inače nema opravdanja za dodanu složenost.

5.1 Prostorni baseline (nearest, bilinear, bicubic, FSR1)

Prva stepenica ne pokušava biti dobra — svrha joj je postaviti donju granicu ispod koje temporalni pristupi ne smiju pasti, i pokazati zašto je vremenska komponenta uopće potrebna. Implementirana su četiri prostorna filtra: najbliži susjed i bilinearano uzorkovanje (hardverski filtri GPU-a), bicubic (Catmull-Rom, odnosno Mitchell-Netravali s parametrima $B = 0, C = 1/2$) i AMD FSR1 (EASU + RCAS).

FSR1 je prostorni upscaler bez povijesti. EASU (*Edge-Adaptive Spatial Upsampling*) prolaz koristi 12 tapova (4×4 bez kutova): četiri preklapajuća petotapna križa, težinski po bilinearnim težinama kvadranta, daju procjenu smjera i jačine ruba; kernel se zatim rotira u smjeru ruba i rasteže okomito na njega, s prozorskom funkcijom koja ga ograničava i deringing korakom koji izlaz stegne u raspon četiri najbliža tupa. RCAS (*Robust Contrast Adaptive Sharpening*) koji slijedi računa 3×3 križ, po kanalu min/max prsten, i ograničava negativnu laticu preostalim prostorom do crne i do bijele boje, skaliranu faktorom 2^{-s} gdje je s parametar oštine.

Objekte implementacije su vlastite, izvedene iz objavljenog opisa algoritma (`ffx_fsr1.h` se čita kao referenca, ne kopira — vidi princip u odjeljku o referentnim implementacijama).

Bitna implementacijska odluka je **redosljed tone mappinga i upscalera**: FSR1 je definiran nad display-referred vrijednostima u $[0, 1]$ — RCAS-ov limiter pretpostavlja da je vrh skale 1, a EASU-ova detekcija rubova pretpostavlja perceptivni razmak između vrijednosti. Renderer radi u linearnom HDR-u, pa lanac ide:

linearni HDR (G-buffer) → tone mapping → upscaler (EASU → RCAS) → prikaz

Suprotan redosljed bi EASU-u dao pogrešne smjerove rubova i RCAS koji ne oštiri ništa.

Rezultati. Mjereno na sceni Sponza, 1920×1080 , 120 okvira, skriptirana kamera, referenca supersamplirana 2×2 i usrednjena u linearnom HDR-u prije tone map-

pinga (razlog je objašnjen u Poglavlju 9):

Upscaler	Quality $1,5\times$ (PSNR / SSIM)	Performance $2,0\times$ (PSNR / SSIM)
Nearest	32,15 dB / 0,8966	31,59 dB / 0,8794
Bilinear	35,25 dB / 0,9333	33,11 dB / 0,8895
Bicubic	35,37 dB / 0,9420	33,13 dB / 0,8997
FSR1 (EASU+RCAS)	33,09 dB / 0,9117	31,92 dB / 0,8785
EASU sam (RCAS isključen)	35,42 dB / 0,9406	—

Tablica 5.1: Prostorni upscaleri, Sponza 1080p.

Zanimljiv je nalaz da je **RCAS oština monotono štetna** na ovoj sceni: ablacija parametra oštine od najjačeg (0) do praktički isključenog (8) daje i PSNR i SSIM koji rastu monotono kako se oštrenje gasi (30,89 dB pri 0 do 35,42 dB pri 8). Optimum je, dakle, degenerirani slučaj u kojem RCAS ne radi ništa — EASU je već izvukao svu informaciju koju jedan okvir sadrži, pa je oština ovdje perceptivni parametar, ne parametar vjernosti, i tako ju treba i izvještavati (ablacijom, ne jednom brojkom). Ovo je izravna posljedica scene: Sponza ima mipmapirane teksture s malo aliasinga za popraviti.

Po cijeni, FSR1 na ovoj sceni **ne ubrzava ništa**: cijena upscalera je fiksna i vezana za izlaznu razlučivost, dok je ušteda vezana za ulaznu — Sponza se na testnom GPU-u renderira native u ≈ 1 ms na 1080p, pa ušteda od nižeg ulaznog uzorkovanja ne pokriva cijenu EASU+RCAS-a i tone mappinga. Ovo je stvarno ograničenje mjerne postavke (scena je jednostavnija od realnog opterećenja igre) i tako je i navedeno u raspravi (Poglavlje 11).

Zaključak koji motivira sljedeći korak: nijedan prostorni filter ne može rekonstruirati informaciju koje u jednom ulaznom okviru nema — razlika među njima je samo koju vrstu pogreške svaki radi. Tek akumulacija kroz vrijeme donosi nove uzorke.

5.2 Minimalni temporalni upscaler (TAAU)

Drugi korak prvi put stvarno *rekonstruira* razlučivost umjesto da je interpolira: kamera svaki okvir pomiče projekciju za pod-pikselni Halton(2,3) pomak, i taj niz pomaka jest dodatna informacija koju nijedan prostorni filter ne može izmisliti. Implementacija je namjerno minimalna — tri dijela: reprojekcija, akumulacija i clamp. Dilatacija vektora, odbacivanje po dubini, lockovi i Lanczos rekonstrukcija dolaze u punom upscaleru (§5.3).

Reprojekcija. Povijest (prethodni izlaz upscalera, u prikaznoj razlučivosti) dohvaća se na $uv - mv(uv) +$, gdje vektor gibanja dolazi iz G-buffera računat iz *nejitriranih* clip-pozicija dok se rasterizacija odvija s jitriranom projekcijom, i čita se isključivo točkastim uzorkovanjem (filtrirani vektor na rubu objekta je prosjek dvaju nepovezanih gibanja i pokazuje nikamo). Povijest se resamplira Catmull-Rom filterom (9 bilinearnih dohvata, standardna faktorizacija 4×4 jezgre s $a = -0.5$), što je mjereno bolje od bilinearnog resamplinga za 0,75 dB u gibanju.

Akumulacija. Za svaki prikazni piksel uzima se 3×3 okolina render piksela oko trenutne uzorkovane pozicije i teži se Gaussovom jezgrom $\exp(-kd^2)$; rezultat se miješa s povijesti težinom koja raste do gornje granice broja okvira. Širina jezgre je bio prvi stvarni nalaz razvoja: uska jezgra ($k \approx 6-8$) čini svaki pojedinačni okvir šumovitim, a taj šum je upravo Halton uzorak koji se prosjekom pretvara u razlučivost — sužavanje jezgre je vrijedilo $+0,5$ dB.

Clamp. Povijest izvan lokalnog raspona boja tekućeg okvira povlači se natrag u njega, u granicama srednje vrijednosti $\pm \gamma$ standardnih devijacija 3×3 okoline. Drugi stvarni nalaz razvoja: clamp mora biti u **YCoCg** prostoru boje i mora biti *clip* (povlačenje po pravcu prema središtu kutije), ne nezavisan clamp triju RGB kanala — nezavisno ograničavanje kanala pomiče ton (na svijetlom kamenu s tamnom fugom povuče crvenu a ne plavu, i piksel ispadne magenta). Prelazak je vrijedio $+1,1$ dB i uklonio vidljiv obrub.

Zašto TAAU u brzom gibanju gubi od bicubica. Ovo je glavni analitički nalaz koraka, ne bug. Interpolirajući filter za resampliranje ima nulu na Nyquistovoj frekvenciji kad pogodi pol-pikselnu fazu; okvir gibanja koji nije cijeli broj piksela zato iz povijesti skida najfiniji detalj. Kako se povijest svaki okvir vraća u akumulaciju s težinom $\approx (1 - 1/f)$, gubitak po okviru postaje stacionaran:

$$G_{\text{stac}} \approx \frac{a}{1 - (1 - a)r}, \quad a = \frac{1}{f}, \quad (5.1)$$

gdje je a udio novog okvira u akumulaciji a r prijenos filtra na Nyquistu za tu fazu. Za $f = 8$ i $r = 0,5$ formula (5.1) daje $G_{\text{stac}} \approx 0,22$ — više od dvije trećine najviših frekvencija nestane, iako svaki pojedinačni resample gubi “samo” pola. Zato duga povijest izgleda oštro dok kamera stoji i omekša čim se pomakne.

Posljedica za dizajn je **skraćivanje povijesti s gibanjem**: gornja granica akumulacije pada eksponencijalno s brzinom gibanja u slici. Ablacija tog mehanizma (Sponza, 60 fps orbita) pokazuje pomak od 2,5 dB između isključenog i uključenog mehanizma — najveći pojedinačni pomak u cijeloj ablaciji ovog koraka — po cijenu temporalne stabilnosti, koja se kreće u suprotnom smjeru: fiksna granica akumulacije daje najstabilniju sliku u tablici, i ujedno najlošiju vjernost, jer ta slika jednostavno ne prati scenu.

Rezultati. Sponza, $1280 \times 720 \rightarrow 1920 \times 1080$ (Quality $1,5 \times$), referenca supersamplirana:

Kamera	Bicubic	TAAU	TAAU + RCAS
mirna	35,83 / 0,9441	39,36 / 0,9723	40,39 / 0,9819
120 fps orbita	37,18 / 0,9596	37,50 / 0,9643	37,71 / 0,9690
60 fps orbita	35,37 / 0,9420	35,13 / 0,9449	35,25 / 0,9505
30 fps orbita	35,27 / 0,9392	34,36 / 0,9372	34,40 / 0,9421

Tablica 5.2: TAAU naspram bicubica, po brzini kamere (PSNR dB / SSIM).

Na mirnoj i sporo pomičnoj kameri TAAU+RCAS dobiva $+4,56$ dB odnosno $+0,53$ dB nad najboljim prostornim filtrom — akumulacija radi ono zbog čega

postoji. SSIM je bolji na svakoj brzini, uključujući one na kojima PSNR zaostaje: to je razlika u strukturi, ne u prosječnoj grešci — bicubic na brzjoj kameri ima puzajući aliasing na rubovima koji SSIM detektira, a PSNR usrednji. Kriterij prihvaćanja iz plana rada (“slika stabilna, vidljiv dobitak PSNR-a nad bicubic”) je ispunjen.

Zanimljivo je da se **RCAS kod TAAU-a ponaša suprotno** nego kod FSR1-a: postoji pravi optimum oštine ($\approx 1,0$ – $1,5$) iznad isključenog RCAS-a (+0,13 dB, +0,0057 SSIM na 60 fps; +1,03 dB na mirnoj kameri), jer izoštravanje ovdje vraća Nyquist koji je resample povijesti dokazivo odnio, a ne izmišlja detalj kojeg nema — suprotno FSR1-u, gdje EASU već izvlači sve što jedan okvir sadrži.

5.3 Puni upscaler (FSR3-stil)

Treći korak dodaje mehanizme koji minimalni TAAU čine upotrebljivim u proizvodnom smislu: dilataciju vektora gibanja, odbacivanje povijesti po dubini (disokluzija), lockove za tanke detalje, adaptivnu Lanczos rekonstrukciju i reactive masku. Implementiran je kao tri compute prolaza: dilatacija i depth-clip zajedno (dijele istu 3×3 okolinu dubine), detekcija lockova, i glavni prolaz koji spaja Lanczos rekonstrukciju, reprojekciju, upravljanje lockovima, rektifikaciju boje i akumulaciju.

Dilatacija vektora. Vektor gibanja vrijedi za površinu koju piksel stvarno pokriva; na silueti render piksel pokriva dvije površine a sprema samo jednu. Uzimanje vektora *najbliže* dubine u 3×3 prozoru pomiče svaki rubni piksel prema prednjoj (vizualno dominantnoj) površini.

Depth clip. G-buffer po pikselu piše i ovokvirnu i prethodnu linearnu udaljenost iste površine (oboje besplatno uz reverse-Z projekciju s beskonačnom dalekom ravninom, jer je $\text{clip.w} = -\text{viewZ}$ egzaktno). Ako vektor pokazuje na površinu koja je stvarno bila tamo prošli okvir, dubinski spremnik na toj poziciji odgovara; ako ne, piksel je bio skriven i njegova povijest je tuđa boja. Prag tolerancije je relativan i uvećan za lokalni nagib dubine, da se pod gledan pod ostrim kutom ne proglasi lažno disokludiranim.

Lockovi. Rektifikacija boje neselektivno povlači povijest prema susjedstvu tekućeg okvira, što je kobno za tanke detalje (karika lanca, stabljika, rub draperije) koji na neke Halton faze jednostavno ne padnu na render uzorak: clamp tada briše akumulirani dokaz da detalj postoji i slika treperi u ritmu jitter sekvence. Detekcija (po uzoru na FSR2) traži piksel koji je strogi ekstrem luminancije duž barem jednog od četiri smjera kroz 3×3 prozor — linija široka jedan piksel je ekstrem *poprijeko*, a ne *uzduž*, što ga razlikuje od šuma (ekstrem u svakom smjeru) i od ruba (ekstrem u nijednom). Lock živi ograničen broj okvira, putuje s poviješću i dok traje širi clamp kutiju te joj briše presjek s tvrdim min/max — jer je upravo taj presjek operacija koja briše detalj.

Lanczos rekonstrukcija prilagođena gibanju. Umjesto Gaussove jezgre iz TAAU-a, glavni prolaz koristi Lanczos-2 preko 4×4 render uzoraka. Širina jezgre vezana je na istu mjeru gibanja kao i duljina akumulacije:

```
stillness    = exp(-motionDecay * mvPixels);
```

```
maxWeight    = mix(1.0, maxWeightConst, stillness);
lanczosScale = mix(lanczosScaleMoving, lanczosScaleStill, stillness);
```

Uska jezgra trguje oštrinu jednog okvira za njegov šum, što je bezopasno samo dok ima dovoljno okvira da se usrednji. Izmjereni optimum na mirnoj kameri je 2,0, u brzom gibanju 1,0.

Rezultati. Ista scena i postavka kao TAAU:

Kamera	TAAU (prethodni korak)	FSR (ovaj korak)	Δ
mirna	39,36 / 0,9723	40,06 / 0,9828	+0,70 dB
120 fps	37,50 / 0,9643	37,53 / 0,9636	+0,03 dB
60 fps	35,13 / 0,9449	35,48 / 0,9460	+0,35 dB
30 fps	34,36 / 0,9372	34,95 / 0,9388	+0,59 dB

Tablica 5.3: Puni upscaler naspram TAAU, po brzini kamere.

Najvažnija posljedica: na 60 fps orbiti puni upscaler **prvi put nadmašuje bi-cubic i po PSNR-u i po SSIM-u** (35,48/0,9460 naspram 35,37/0,9420 iz §5.1) — minimalni TAAU je tamo još zaostajao. Temporalna stabilnost se pri tom popravlja time što se *približi* referenci, ne time što raste: na 60 fps orbiti referenca ima stabilnost 34,00 dB, TAAU 36,94 (+2,94, previše stabilno — akumulacija prigušuje i legitimne promjene), a puni upscaler 34,15 (+0,15, ponaša se temporalno kao ground truth).

Ablacija po mehanizmu (60 fps orbita naspram mirne kamere) pokazuje da **dilatacija donosi najveći pojedinačni dobitak, i to na mirnoj kameri** (+2,50 dB) — naizgled kontraintuitivno, jer se kamera ne miče, ali baš zato je povijest najdulja i pogrešan vektor na silueti akumulira grešku kroz svih osam okvira umjesto da je jednom napravi. Depth clip je mjerljivo koristan samo u gibanju (+0,13 dB), a lockovi rade suprotno: -0,06 dB u gibanju, +0,51 dB na miru.

Dva mehanizma iz nacrtala dala su **negativan ili nulti rezultat, i oba su izvještena kao takva**: reactive maska monotono šteti (Sponza nema pravu prozirnost — jedine nekontinuirane površine su alfa-testirano lišće, čiji je alfa test deterministički, pa je povijest koju maska baca bila točna), a RCAS na izlazu punog upscalera sad šteti umjesto da pomaže, jer je adaptivna Lanczos jezgra i lockovi već uklonili nedostatak (izgubljeni Nyquist) koji je RCAS kod TAAU-a popravljao.

Nedostaje luma piramida iz izvornog nacrtala (FSR2 je koristi za auto-ekspoziciju) — ovaj pipeline tone-mapa *prije* upscalera, pa je ekspozicija već fiksirana i piramida ne bi promijenila ništa; izostavljena je iz istog razloga i reverzibilni interni tonemap. Oba su namjerna odstupanja od nacrtala, ne propust, i raspravljena su u Poglavlju 11.

6 | Implementacija procjene gibanja

Upscaler iz prethodnog poglavlja koristi vektore gibanja koje daje rasterizacija — poznati su egzaktno za svaki trokut koji se crta. Oni, međutim, ne opisuju sjene, refleksije, promjene sjenčanja, animirane teksture niti sadržaj koji se ne renderira po trokutima (UI, čestice). Procjena gibanja *iz same slike* — optical flow — postoji upravo za taj ostatak, i to je jezgra “napredne metode računalnog vida” u ovom radu. Koristi ga isključivo generiranje okvira (Poglavlje 7); upscaler ga po zadanom ne koristi, da ne dodaje svoju cijenu svakom mjerenju upscalera.

6.1 Piramidalni block matching

Implementacija procjenjuje gibanje između prikazanog i prethodnog prikazanog okvira, za svaki blok od 8×8 texela luminancije (na pola prikazne razlučivosti), po uzoru na `FfxOpticalFlow` iz FidelityFX SDK-a. Vektori su, po konvenciji cijelog projekta, u UV prostoru i pokazuju iz trenutnog okvira u prethodni — identično polju gibanja iz G-buffera, čime su dva polja izravno usporediva i zamjenjiva.

Glavna petlja radi coarse-to-fine, 7 razina piramide, tri prolaza po razini:

1. **Pretraga** (*search*) — za svaki nepreklapajući blok od 8×8 texela, pretraga prozora radijusa R oko predikcije iz grublje razine, kriterij minimalna suma apsolutnih razlika (SAD) luminancije.
2. **Filtriranje** (*filter*) — 3×3 vektorski medijan (vektor s minimalnom sumom udaljenosti do preostalih osam) uklanja izolirane ispade.
3. **Profinjenje** (*upscale*) — prijelaz na finiju razinu bira između kandidata (roditelj i tri susjeda) po SAD-u; ne izvodi se na zadnjoj razini.

Vektori se u ovoj implementaciji nose u UV prostoru *pune slike* na svakoj razini piramide, ne u pikselima razine, čime prijelaz između razina postaje čist izbor kandidata bez množenja s dva — cijela klasa grešaka “u kojoj je razini ovaj vektor” ne može nastati. Cijena je odsutnost sub-pixel profinjenja: najsitniji razlučiv pomak je jedan texel luminancije na pola razlučivosti, tj. dva prikazna piksela.

Kriterij podudaranja je SAD, a ne SSD (suma kvadrata razlika): kvadriranje bi jednom specularnom pikselu s velikom promjenom dalo moć da poništi inače savršeno podudaranje. SAD se normalizira u srednju apsolutnu razliku po texelu, uz kaznu udaljenosti od predikcije (kazna glatkoće) u istim jedinicama — bez toga bi pretraga unutar ravne plohe (gdje svaki kandidat postiže isti SAD) birala slučajnog pobjednika umjesto grublje razine kao prirodnog prior-a.

6.2 Referenca i točnost

Optical flow u pravom slučaju uporabe nema ground truth. Ali dok je jedino gibanje gibanje kamere u statičnoj sceni (Sponza), vektori iz rasterizacije su egzaktni i postaju besplatna referenca. Pogreška se mjeri kao endpoint error (EPE) u prikaznim pikselima — euklidska udaljenost između procijenjenog vektora i prosjeka referentnog polja preko površine bloka.

Kamera	EPE sred.	EPE medijan	EPE p95	unutar 1 px	unutar 2 px
mirna	0,00	0,00	0,00	100,0%	100,0%
120 fps	2,82	1,49	7,62	65,7%	89,0%
60 fps	11,63	3,66	36,81	56,9%	80,2%
30 fps	35,79	8,83	106,72	47,5%	70,4%

Tablica 6.1: Točnost procjene gibanja po brzini kamere.

Za usporedbu, procjenitelj koji uvijek vrati nulti vektor daje srednju pogrešku 23,0 / 46,1 / 92,3 px pri 120/60/30 fps — implementirani procjenitelj dakle uklanja $\approx 87\%$ gibanja pri 120 fps, 75% pri 60 fps i 61% pri 30 fps. Degradacija s brzinom nije linearna jer block matching ima tvrd doseg: radijus R na razini L pokriva $2^L \cdot R$ texela, a preko toga se podudaranje ne pogoršava postupno nego se posve izgubi.

6.3 Ablacije

Tri mehanizma doprinose na različit način, mjereno na istoj orbiti (60 fps, sve ostalo zadano):

Konfiguracija	EPE sred.	medijan	p95	unutar 1 px
sve zadano	7,17	1,36	33,50	62,5%
bez kandidata iz prošlog okvira	11,04	1,84	49,96	60,1%
bez medijan filtra	7,95	2,39	34,64	56,5%
bez izbora kandidata pri profinjenju	12,68	1,18	56,98	61,5%

Tablica 6.2: Ablacija mehanizama procjene gibanja.

Temporalni kandidat (polje prošlog okvira kao dodatni kandidat u pretrazi) je najveći pojedinačni doprinos srednjoj pogrešci — gibanje je vremenski koherentno, pa je prošlo polje bolji prior od bilo čega što piramida sama može ponuditi, i ne degradira s veličinom pomaka kao piramida. **Medijan filtar** se vidi u medijanu i udjelu točnih blokova, a jedva u srednjoj vrijednosti — uklanja izolirane ispade, ne velike sistematske promašaje. **Izbor kandidata pri profinjenju** je obrnut slučaj: bez njega je medijan bolji, ali p95 naraste sa 33,5 na 57,0 — slijepo kopiranje roditelja razmazuje granice grubljih blokova u finiju razinu.

Radijus pretrage je odabran kao kompromis točnosti i cijene: od 4 do 8 texela točnost se popravi 9% uz 46% skuplji modul, pa je 4 zadano.

6.4 Detekcija promjene scene

Nakon reza dva uzastopna okvira nemaju nikakve veze, a block matching bi svejedno vratio “najbolje” podudaranje, koje je smeće. Detektor uspoređuje histograme luminancije po devet sekcija slike, računa udaljenost totalne varijacije po sekciji i iz devet brojeva izvodi jednu presudu. Mjereno na tri orbite s umjetno umetnutim rezovima, statistika koja se uzima nad devet sekcija odlučuje o pouzdanosti:

Statistika	min(rez)	max(gibanje)	detektirano	lažnih pozitiva
maksimum	0,568	0,463	15/15	11/105
srednja	0,253	0,207	15/15	0/105
medijan	0,232	0,179	14/15	0/105

Tablica 6.3: Statistika detekcije reza scene, mjerna orbita.

Maksimum ima najširu marginu i ipak je najgori izbor: brzi zaokret kamere zamijeni sadržaj nekoliko sekcija na vodećem rubu, što se od stvarnog reza ne razlikuje ni po čemu drugom, i proizvodi lažne pozive. Srednja vrijednost hvata sve rezove uz nula lažnih poziva na dvije od tri testirane orbite i zadana je statistika.

6.5 Cijena i granice

Modul košta $\approx 0,56$ ms na 1080p (720p $\rightarrow$ 1080p upscaling) odnosno $\approx 21\%$ cijene okvira; najfinija razina piramide je 41% te cijene, šest grubljih razina zajedno manje od trećine. Poznata ograničenja, izmjerena i prihvaćena kao svojstvo metode: periodične teksture (fino tkanje, prugasti stupovi) daju bloku mnogo jednako dobrih podudaranja pa je procjena tamo šumovita (aperture problem u Fleet–Weiss formulaciji); ravne plohe bez strukture nemaju jedinstven minimum i procjena pada na prior; a blok koji prekriva siluetu s dva gibanja mora izabrati jedno, što je granica rezolucije mreže bloka, ne greška algoritma.

Teorijska podloga je Fleet i Weiss, *Optical Flow Estimation* [1]: pretpostavka konstantnosti svjetline, aperture problem i loša postavljenost problema unutar homogenih područja — tri pojave koje ablacije u ovom poglavlju izravno mjere.

7 | Implementacija generiranja okvira

Zadnji dio klasičnog cjevovoda iz dva uzastopna upscalana okvira, vektora gibanja iz G-buffera i polja optical flowa (Poglavlje 6) gradi okvir na trenutku $t - 0,5$ — onaj koji bi se prikazao između njih — i stavlja ga na ekran u pravom trenutku. Ovo poglavlje pokriva izgradnju međuokvira (Modul D iz plana) i njegov prikaz: frame pacing, latenciju i kompoziciju korisničkog sučelja.

7.1 Izgradnja međuokvira

Konvencija je ista kao u cijelom projektu: vektori pokazuju iz trenutnog okvira u prethodni. Površina koja se između $t - 1$ i t pomaknula za vektor $\vec{v}$ bila je na pola puta u $t - 0,5$, pa se međuokvir iz trenutnog čita na $uv - 0,5\vec{v}$, a iz prethodnog na $uv + 0,5\vec{v}$.

Postupak ide u devet compute prolaza:

1. **Postavljanje** — pražnjenje ciljnih slika (prioritet 0).
2. **Dubina međuokvira** — scatter dubine na $uv + 0,5\vec{v}$, atomski minimum.
3. **Polje game vektora** — scatter vektora iz G-buffera s prioritetom.
4. **Piramida polja** — popunjavanje rupa u $\log(n)$ koracima (zajednička za oba polja).
5. **Polje optical flow vektora** — isti scatter, iz procjene gibanja.
6. **Maske disokluzije** — dvije maske, međuokvir $\leftrightarrow$ prethodni i međuokvir $\leftrightarrow$ trenutni.
7. **Interpolacija** — dva warpa po polju, izbor između polja po slaganju boja; alfa kanal = pokrivenost.
8. **Piramida za inpainting (M8)** — piramida slike množene pokrivenošću.
9. **Inpainting (M8)** — popunjavanje preostalih rupa iz piramide.

Zašto scatter i atomici. Polje vektora međuokvira ne postoji u nijednom ulazu — svaki izvorni piksel piše tamo gdje je njegova površina u $t - 0,5$, i više piksela može pasti na isti texel odredišta. Pobjednik se bira bez drugog prolaza kroz kandidate pomoću `imageAtomicMax` nad 32-bitnom riječi čiji gornjih 16 bita nosi prioritet a donjih 16 komponentu vektora kao *half float* — atomik uspoređuje prvo prioritet, vrijednost tek kao razrješenje izjednačenja. Prioritet game polja kodira (od

najznačajnijeg bita): je li površina najbliža na tom texelu, dubinu (bliže = više) i slaganje boje na kraju vektora; prioritet polja toka kodira umjesto dubine sigurnost podudaranja bloka i magnitudu pomaka.

Popunjavanje rupa u piramidi zadržava, pri redukciji, dijete s *najnižim* prioritetom (pozadinu): piksel u disokluziji pripada onome što je bilo *iza* objekta koji se odmaknuo, i to opisuje vektor pozadine, ne vektor prednje površine.

Interpolacija radi dva izbora: unutar polja (koji od dva warpa vrijedi — određuju maske disokluzije) i između polja (kojem vektoru vjerovati — određuje slaganje boja na krajevima warpa, s težinom $\exp(-k \cdot \text{udaljenost boja})$; točan vektor dovede istu površinu na samu sebe i oba warpa se slažu, pogrešan se sam prijavi razlikom).

7.2 Rezultati

Sve mjereno na Sponzi, referenca je pravi render trenutka $t - 0,5\Delta t$ (supersampliran, bez jittera), a stupac “blend” je naivni 50/50 blend istog para ulaznih okvira naspram iste reference — cijena nečinjenja i jedina poštena usporedba (ista razlučivost, isti upscaler, isti trenutak).

Konfiguracija	PSNR (min)	PSNR blend	dobitak
1080p Quality	34,94 (27,56)	26,11	+8,83 dB
1080p native	36,43 (27,92)	26,14	+10,29 dB
1080p Performance	33,52 (27,51)	26,16	+7,36 dB

Tablica 7.1: Interpolirani okvir naspram 50/50 blenda, po razlučivosti render prolaza.

Dobitak nad blendom je 7–10 dB na svakoj razlučivosti; kvaliteta međuokvira prati kvalitetu ulaza (native vodi, Performance zaostaje). Po brzini kamere, mirna kamera je provjera ispravnosti (blend je egzaktna, modul ga ne kvari do stotinke), a srednji dobitak je gotovo neovisan o brzini gibanja (+8 do +9 dB), ali **najgori okvir** nije: pri 30 fps pada na razinu prosjeka blenda, pri 20 fps ispod nje — pretpostavka linearnog gibanja i sve šire disokluzije tamo prestaju vrijediti.

Ablacija na dvije scene (Sponza, gdje su game vektori egzaktni, i proceduralna scena s pomičnim objektima) pokazuje da se **dva polja nadopunjuju**: samo game vektori daju 34,80/33,68 dB (Sponza/proceduralna), samo optical flow 33,74/33,53 dB, a oba zajedno 34,94/34,07 dB — nijedno polje samo ne doseže kombinaciju. Maske disokluzije donose malen ali mjerljiv dobitak (+0,09 dB na Sponzi), dok dilatacija vektora i izbor djeteta u piramidi nemaju mjerljiv učinak na ovim scenama (rupe u polju su preuske da bi izbor susjeda promijenio vektor) — što je i samo rezultat: mehanizmi postoje kao u FSR3, na ovim scenama nemaju što popraviti.

7.3 Prikaz: frame pacing, latencija, UI

Interpolirani okvir mora stići na ekran u pravom trenutku, ne odmah kad je gotov. Tri su pitanja, sva o vremenu i redoslijedu, ne o slici: *kada* se prikazuje koji okvir

(pacing), *koliko kasni* ono što igrač vidi za onim što je napravio (latencija), i *gdje u lancu* ulazi korisničko sučelje.

Arhitektura. Render nit radi na skrivenom 1×1 prozoru; zaseban nit prikaza, s vlastitim GL kontekstom koji dijeli teksture, brine se o trenutku prikaza. Oba okvira para (generirani i pravi) crtaju se u back buffer odmah pri predaji (*staging*), pa u trenutku pravog prikaza preostaje samo swap. Prva verzija (blit neposredno prije swapa) davala je okvire koji su i dalje stizali u parovima, jer dva konteksta na istom GPU-u ne mogu preskočiti red poslova — staging to zaobilazi predajom oba bliteta odmah dok je GPU prazan.

Rezultati pacinga. Sponza, sintetsko opterećenje (dodatni G-buffer prolazi po okviru simuliraju cijenu igre):

Opterećenje	bez FG	FG ravnomjerno	ubrzanje
$\times 0$	313,0	308,3	$0,98\times$
$\times 4$	155,3	211,0	$1,36\times$
$\times 12$	90,7	135,3	$1,49\times$
$\times 24$	57,7	89,9	$1,56\times$

Tablica 7.2: Prikazani FPS naspram sintetskog opterećenja.

Ispod praga isplativosti generiranje ne donosi ništa ($\times 0$: FPS ostaje na mjestu, jer je cijena generiranja bliska cijeni renderiranja lagane scene), a iznad njega ubrzanje raste na $1,5\text{--}1,6\times$ — nikad na $2\times$, jer generiranje nije besplatno i uvijek renderirani FPS uz njega padne. **Ravnomjernost** pacinga je mjerena standardnom devijacijom razmaka između prikazanih okvira: bez pacinga 7,03 ms (bimodalno, okviri stižu u parovima), s ravnomjernim pacingom 1,00 ms — manje nego bez generiranja uopće (1,38 ms).

Latencija raste za $\approx$ jedan render period naspram okvira bez generiranja (npr. +11 ms uz $\times 12$, +16 ms uz $\times 24$) — to je cijena tehnologije, mjerena i izvještena, ne pretpostavljena.

UI kompozicija. HUD komponiran *nakon* generiranja (na čistoj slici bez teksta) naspram upečenog *prije* njega (kao dio scene koju warp pomiče): razlika je 32 dB u pravokutniku HUD-a i 4,4 dB na cijelom okviru — upečeni HUD gubi više kvalitete nego sve ablacije generiranja zajedno, jer warp tretira tekst kao dio scene i udvostruči ga.

Cijena čitavog generiranja okvira (uz optical flow) na 1080p Quality je $\approx 2,2$ ms, što je usporedivo s cijenom jednog pravog okvira ove testne scene — otud prag isplativosti u tablici gore, i otud napomena da je svaka scena teža od testne iznad tog praga.

8 | Naučena komponenta

Opis teme traži i naučenu komponentu, a FSR3 je u cijelosti klasičan računalni vid — naučeni komercijalni pristupi (DLSS, FSR4 “Redstone”) žive na hardveru s matričnim jedinicama koje razvojni GPU ovog rada nema (Poglavlje 3). Ova komponenta zato ne zamjenjuje jezgru cjevovoda, nego mjeri jedno izolirano pitanje: **može li mala mreža, uz iste ulaze, bolje od ručne heuristike odlučiti kako spojiti kandidate koje klasični lanac (Poglavlje 7) već proizvodi — i koliko to košta na GPU-u bez podrške za tenzore.**

8.1 Što mreža radi

Mreža **ne crta piksele**. Za svaki piksel predviđa softmax težine nad sedam kandidata boje koje klasični dio generiranja okvira već ima: izlaz heuristike, warp igrinim vektorima iz $t - 1$ i iz t , warp optical flowom iz $t - 1$ i iz t , te trenutni i prethodni okvir bez warpa. Izlaz je njihov ponderirani zbroj. Tri razloga za takav oblik:

1. **Čista usporedba.** Ulazi i kandidati su identični heuristici; naučeno je samo pravilo miješanja, pa je razlika u PSNR-u razlika pravila, ne dodatne informacije.
2. **Ne može halucinirati.** Izlaz je konveksna kombinacija postojećih boja, pa mreža ne treba kapacitet za sintezu detalja — staje u nekoliko tisuća parametara i u compute shader.
3. **Počinja od heuristike.** Pristranost zadnjeg sloja je inicijalizirana tako da kandidat 0 (heuristika) dobije $\approx 90\%$ težine; svaki odmak od klasičnog rezultata učenje mora zaraditi na podacima.

8.2 Arhitektura

Mala U-Net-lite mreža (odabrani model, oznaka c8-c16, ima 7199 parametara) radi u compute shaderima na 20 značajki po pikselu (valjanost i pokrivenost polja, slaganje boja među kandidatima, udaljenosti heuristike od warpova, razine piramide, duljine vektora, luminancija). Arhitektura ima enkoder od tri razine (pooling 2×2 + konvolucija 3×3) i dekoder s preskočnim vezama, završava 1×1 slojem i softmaxom. Kanali žive četiri po RGBA16F sloju teksturnog polja, pa je

3×3 konvolucija $16 \rightarrow 16$ kanala oblik posla ($4 \times 9 \times 4$ umnožaka $\text{mat4} \cdot \text{vec4}$ po pikselu) u kojem su i GPU-ovi bez matričnih jedinica još razumno dobri.

8.3 Skup podataka i učenje

Skup podataka snima vlastiti renderer u lockstep načinu: svaki mjereni okvir daje patcheve 96×96 značajki i kandidata, uz **stvarno renderirani među-okvir** kao oznaku (isti postupak kao referenca u Poglavlju 7). Podjela na skup za učenje i validaciju je po *putanji kamere*, ne po okviru — uzastopni okviri iste putanje su gotovo ista slika, pa bi podjela po okviru ocjenjivala mrežu na podacima koje je efektivno već vidjela. Mjerni pogledi korišteni za rezultate u ovom radu nikad se ne snimaju u skup za učenje.

Trening koristi vlastiti trener napisan u C++20 s OpenMP-om, na CPU-u, a ne PyTorch na vanjskom GPU-u: model je dovoljno malen (nekoliko tisuća parametara) da su unaprijedni i povratni prolaz nekoliko stotina linija petlji koje prevoditelj vektorizira, a razvojni GPU nema ROCm podršku potrebnu za trening na njemu. Posjedovanje obje strane (trener i shader inferencije) omogućuje i međusobnu provjeru: snimka napravljena s učitanoj mrežom nosi u svakom patchu i izlaz shadera, a trener izračuna vlastiti unaprijedni prolaz i uspoređuje ga s njim — za odabrani model srednja apsolutna razlika je $7,1 \times 10^{-5}$, najveća $4,7 \times 10^{-4}$ (zaokruživanje half-floata). Bez ove provjere greška u redoslijedu stupaca matrice ili u poravnanju poolinga davala bi mrežu koja naizgled radi, a tiho gubi kvalitetu.

8.4 Rezultati

Pogled	Heuristika (PSNR/SSIM)	Mreža (PSNR/SSIM)	Dobitak
1080p Quality	35,00 / 0,9443	35,10 / 0,9458	+0,10 dB
1080p native	36,50 / 0,9628	36,69 / 0,9650	+0,19 dB
proceduralna scena	34,06 / 0,9708	34,26 / 0,9722	+0,20 dB
20 fps	30,95 / 0,9122	31,11 / 0,9181	+0,16 dB

Tablica 8.1: Naučena mješavina naspram heuristike, odabrani mjerni pogledi.

SSIM raste na svih osam mjerenih pogleda, PSNR na sedam od osam (+0,03 do +0,20 dB), stabilno preko tri sjemena treniranja. Dobitak **nije sustavan na najgorem pojedinačnom okviru** (mreža uči prosjek; rijetki okviri s brzim zamahom kamere su u skupu za učenje zastupljeni koliko su i rijetki) i **ne prenosi se na neviđenu scenu**: model treniran samo na jednoj sceni daje na drugoj, njemu neviđenoj sceni, praktički identičan rezultat heuristici.

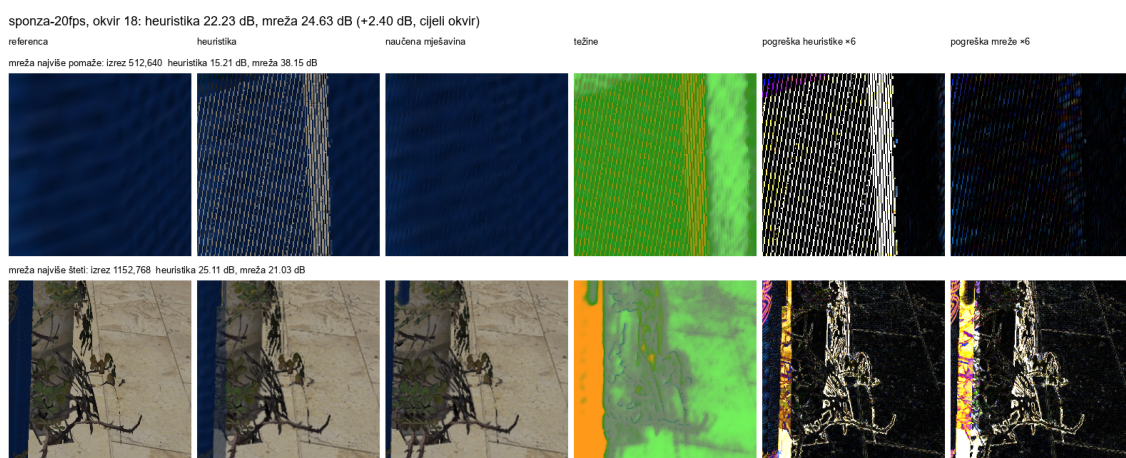
8.5 Cijena

Naučeni dio na 1080p Quality (Sponza) košta 2,7 ms po okviru GPU vremena (generiranje okvira ukupno $1,84 \rightarrow 4,58$ ms). Pri sintetskom opterećenju $\times 12$ (Po-

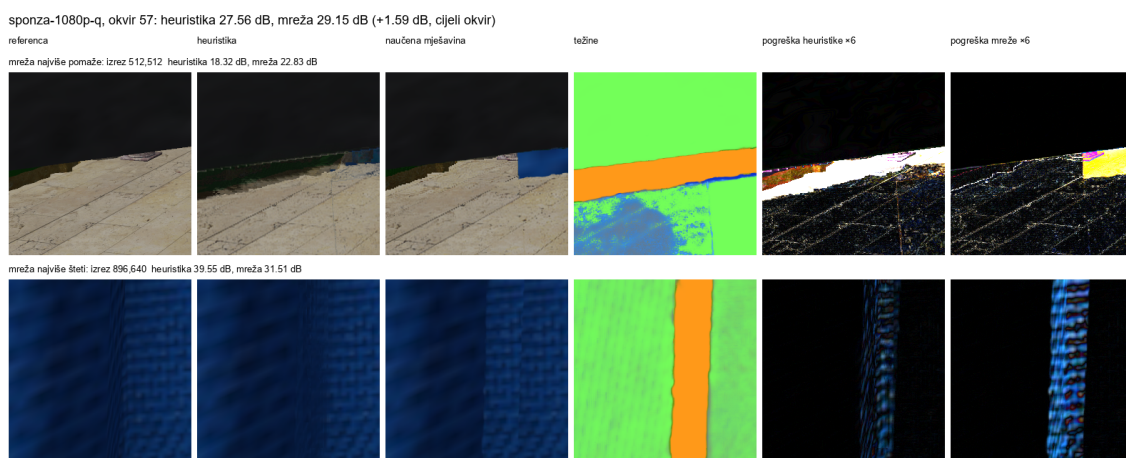
glavljje 7, odjeljak o patingu) prikazani FPS pada s 147 na 120. Na razvojnom hardveru se ne isplati: heuristika je 0,10 dB lošija za 40% cijene generiranja okvira. To je izmjeren, ne pretpostavljen odgovor na pitanje teme: gdje je granica malog naučenog modela bez tenzorskih jedinica.

8.6 Demonstracija

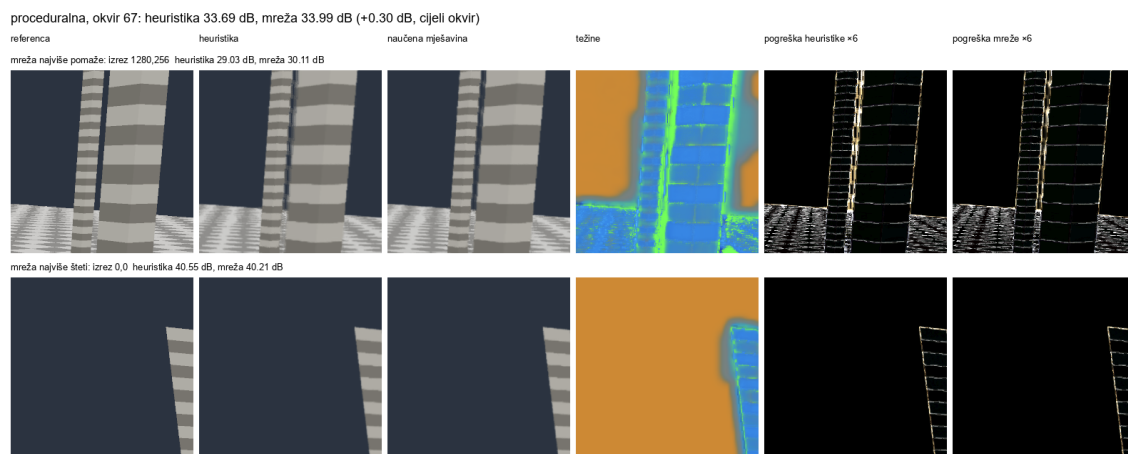
Slika 8.1 pokazuje oba ekstrema u istom okviru (20 fps, najveća disokluzija): u jednom izrezu mreža uklanja prugavi artefakt na rubu (15,21 $\rightarrow$ 38,15 dB), u drugom zamuti sitnu granu lišća (25,11 $\rightarrow$ 21,03 dB). Slika 8.2 pokazuje isti obrazac u uobičajenom radnom režimu (1080p Quality), a Slika 8.3 pokazuje neviđenu scenu, gdje je razlika zanemariva — vizualna potvrda nalaza iz prethodnog odjeljka.



Slika 8.1: 20 fps, okvir 18: najveći dobitak i najveći gubitak mreže u istom okviru. Stupci: referenca, heuristika, naučena mješavina, prikaz težina, pogreška heuristike i mreže ($\times 6$).



Slika 8.2: 1080p Quality, okvir 57: tipičan radni režim.



Slika 8.3: Proceduralna scena (neviđena pri treniranju), okvir 67: razlika zanemarljiva.

Boje prikaza težina

- heuristika
- game warp t-1
- game warp t
- flow warp t-1
- flow warp t
- trenutni okvir
- prethodni okvir

Slika 8.4: Legenda boja prikaza težina iz slika iznad.

9 | Metodologija mjerenja

9.1 Ground truth

Kvaliteta u ovom radu je brojka jer postoji referenca, a referenca postoji zato što vlastiti renderer to dopušta. Za upscaler, referenca je puni okvir u prikaznoj razlučivosti, bez jittera, u trenutku t . Za generiranje okvira, referenca je scena i kamera pomaknuta na $t - 0,5\Delta t$ (bez upisivanja u povijest instanci, da idući pravi okvir ne dobije prekratke vektore gibanja), renderirani bez jittera i vraćeni na t . Oba referentna prolaza renderiraju se supersamplirano (2×2) i usrednjena u linearnom prostoru boje prije tone mappinga — jednostruka referenca u prikaznoj razlučivosti bi bila aliasirana, a bodovanje naspram aliasirane referencije sustavno nagrađuje zamućenje (Poglavlje 2).

Ispravnost među-okvira provjerava se neovisno o glavnom mjernom putu: pomoćna skripta provjerava da je referentni među-okvir podjednako udaljen (PSNR) od oba susjedna pravog okvira i bliži svakom od njih nego što su susjedi jedan drugome. Na testnoj sekvenci asimetrija je 0,01 dB — među-okvir leži na sredini unutar mjerne pogreške. Ovom provjerom dobiva se i donja granica koju generiranje okvira mora nadmašiti da bi imalo smisla: PSNR naivnog 50/50 blenda dva susjedna pravog okvira naspram istog referentnog među-okvira (28,85 dB na testnoj putanji) — donja granica jer i blend i generiranje okvira koriste isti ulazni par, a razlika je što blend ne zna ništa o gibanju.

9.2 Skriptirana kamera i determinizam

Svako mjerenje koristi skriptiranu putanju kamere i fiksni vremenski korak (Δt), umjesto stvarnog proteklog vremena — bez toga dva pokretanja ne bi proizvela istu sekvencu okvira, a referenca koja se ne može ponoviti nije referenca. Način rada s fiksnim brojem okvira ne prihvća unos s tipkovnice ni miša: jedan slučajaj pritisak tijekom mjerenja (npr. razmaknica, koja pauzira scenu) tiho bi promijenio ono što se mjeri bez greške u izvođenju programa. Mjerna skripta zato provjerava da se vrijeme scene stvarno pomiče između uzastopnih okvira mjerenja i prekida mjerenje ako ne — jedina iznimka je mjerenje s namjerno mirnom kamerom, gdje se vrijeme scene pomiče za korak ispod razlučivosti izvještaja.

9.3 Mjerenje performansi

Vrijeme na GPU-u mjeri se *po passu*, GPU timestamp upitima (ne CPU mjerenjem obrađenim naknadno), jer je upravo raspodjela troška po passu ono što rad čini inženjerskim rezultatom, ne demonstracijom. Prvih nekoliko okvira svakog mjerenja (*warmup*) se odbacuje iz izvještaja: brojaču vremena treba nekoliko okvira da počne vraćati broj (prsten upita), a prvi pravi okviri uključuju prevođenje shadera i prve alokacije tekstura, pa mogu biti red veličine skuplji od stabilnog stanja. Prolazi koji postoje samo radi mjerenja (npr. renderiranje dodatne referentne slike uz `--validate-upscale`) mjere se, ali se eksplicitno isključuju iz ukupne cijene okvira koja se izvještava — inače bi mjerenje samo sebe naplaćivalo.

9.4 Mjerenje kvalitete: PSNR i SSIM na GPU-u

PSNR i SSIM računaju se izravno na GPU-u, u compute shaderu, u dvostupanjskoj redukciji: prvi prolaz akumulira djelomične sume po radnim skupinama 16×16 u storage buffer, a CPU tek zbraja te djelomične sume — izravna atomska akumulacija u float preko cijele slike bi izgubila preciznost (velik broj malih dodataka istoj varijabli). SSIM se računa po formulaciji Wanga i sur. [6]: Gaussov prozor 11×11 ($\sigma = 1,5$) na kanalu luminancije, konstante $C_1 = 0,0001$, $C_2 = 0,0009$, u dva separabilna prolaza (vodoravni i uspravni).

Temporalna stabilnost (koliko slika treperi kroz vrijeme, što PSNR/SSIM jednog okvira ne mjeri) izvještava se **relativno naspram iste mjere nad referentnim nizom**, ne naspram nule: promjene sjenčanja, disokluzije i resampliranje same referencije prežive reprojekciju kao i izlaz upscalera, pa referenca nije savršeno stabilna nula. Iznad ≈ 50 dB ova metrika prestaje razlikovati bilo što (RMS razlika $< 0,002$) i u tom području se ne čita.

9.5 Alat za regeneraciju tablica

Svih 283 konfiguracija koje se pojavljuju u ovom radu regenerira se jednom naredbom (`scripts/run_metrics.py`): skripta redom pokreće svaku konfiguraciju, prikuplja CSV po okviru i agregira sve rezultate koji se trenutno nalaze na disku u jednu zbirnu tablicu — bez obzira je li ih pokrenula upravo ta invokacija ili prijašnja. To znači da se cijeli mjerni paket, na bilo kojem računalu, može reproducirati bez ručnog prepisivanja brojki, što je i korišteno za provjeru na dva različita GPU-a u odjeljku 10.1.

10 | Rezultati

10.1 Provjera na drugom hardveru

Cijeli mjerni paket (283 konfiguracije, `scripts/run_metrics.py`) je pokrenut na dva potpuno različita računala, s istim izvornim kodom (isti *commit*), da se provjeri da rezultati ne ovise o jednom proizvođaču GPU-a niti o slučajnosti jedne mjerne postavke.

	Računalo A	Računalo B
GPU	NVIDIA GeForce RTX 5070	AMD Radeon RX 7800 XT (Sapphire Nitro+)
CPU	AMD Ryzen 7 7800X3D	AMD Ryzen 7 5700X3D
Driver	NVIDIA 616.92 (32.0.16.1692)	najnoviji AMD Adrenalin
OS	Windows	Windows

Tablica 10.1: Dvije mjerne postavke korištene za provjeru.

Ispravnost. Sve 283 konfiguracije podudaraju se po imenu između dva skupa rezultata, pa je usporedba red po red. Metrike kvalitete (PSNR, SSIM, endpoint error procjene gibanja) gotovo se ne razlikuju: srednja apsolutna razlika je reda 10^{-3} – 10^{-1} dB odnosno piksela na svim mjerenim stupcima. Najveće pojedinačne razlike (do $\approx 0,7$ – $0,9$ dB) padaju isključivo na konfiguracije koje su već u poglavljima 5 i 6 obilježene kao mjerno osjetljive same po sebi — mirna kamera (PSNR iznad ≈ 40 dB, gdje se metrika više ne razlikuje smisleno, vidi napomenu u odjeljku o temporalnoj stabilnosti) i ekstremna ablacijska postavka RCAS sharpness = 0. Nigdje se ne javlja sustavno odstupanje karakteristično za, primjerice, grešku u redoslijedu operacija ili u pretpostavci o preciznosti pokretnog zareza specifičnoj za jednog proizvođača. Zaključak: implementacija se ponaša deterministički i prenosivo preko granice AMD/NVIDIA, što nije samo po sebi razumljivo — GLSL prevoditelji, redoslijed zbrajanja u sumama i ponašanje teksturnih filtara razlikuju se između proizvođača.

Performanse. Ukupni omjer GPU vremena (RX 7800 XT / RTX 5070) je blizu 1 (medijan 1,07, srednja vrijednost 1,02) — dva su GPU-a na ovom opterećenju vrlo izjednačena — ali raspodjela po modulu pokazuje dosljedan obrazac, ne slučajan šum:

Obrazac je čitljiv arhitekturno: AMD je brži na prolazima s mnogo aritmetike po texelu i malo teksturnih dohvata (block matching u optical flowu, $3 \times 3/4 \times 4$ konvolucije naučene mješavine), a NVIDIA na akumulacijskom prolazu punog upscalera, koji je dominantno vezan na filtrirane teksturne dohvate (Catmull-Rom

Modul	Medijan omjera	Čitanje
Optical flow (M6)	0,85–0,88	RX 7800 XT $\approx 15\%$ brži
Naučena mješavina (M9)	0,88–0,93	RX 7800 XT $\approx 10\%$ brži, više na v
TAAU, jezgra generiranja okvira (M4, M7)	1,00–1,08	praktički izjednačeno
Puni upscaler FSR (M5)	1,08–1,14	RX 7800 XT $\approx 10\%$ sporiji

Tablica 10.2: Omjer GPU vremena po modulu, RX 7800 XT naspram RTX 5070.

i Lanczos uzorkovanje povijesti). Nijedan GPU nije jednoznačno brži na cijelom cjevovodu, što je i razlog zašto se apsolutne brojke u ovom radu izvještavaju po modulu, a ne kao jedna sažeta brojka za “FPS”.

10.2 Sinteza po modulu

Detaljni rezultati i ablacije po modulu, s tablicama i analizom, dani su uz odgovarajuću implementaciju: prostorni baseline, TAAU i puni upscaler u Poglavlju 5, procjena gibanja u Poglavlju 6, generiranje okvira i prikaz u Poglavlju 7, naučena komponenta u Poglavlju 8. Ovaj odjeljak sažima headline brojke na jednom mjestu, redoslijedom kojim se cjevovod nadograđuje.

Korak	Mjera dobitka	Vrijednost
M3 (FSR1 EASU) naspram nearest	PSNR	+3,27 dB
M4 (TAAU+RCAS) naspram M3 bicubic, mirna kamera	PSNR	+4,56 dB
M5 (puni upscaler) naspram M4, 60 fps orbita	PSNR	+0,35 dB
M6 optical flow naspram nulte procjene, 120 fps	udio uklonjenog gibanja	$\approx 87\%$
M7 generiranje okvira naspram 50/50 blenda, 1080p Q.	PSNR	+8,83 dB
M8 pacing naspram trenutnog prikaza, opterećenje $\times 12$	std. razmaka	7,03 $\rightarrow$ 1,00 ms
M9 naučena mješavina naspram heuristike	SSIM	+ na 8/8 pogleda

Tablica 10.3: Headline dobitak svakog koraka cjevovoda naspram prethodnog.

Svaki korak nadmašuje prethodni na istoj sceni i istoj metrici — kriterij koji je u planu rada postavljen kao uvjet prihvatanja za svaki milestone. Iznimka koja se mora navesti otvoreno: M9 pomiče kvalitetu (SSIM raste na svih osam mjerenih pogleda), ali ne pomiče omjer kvalitete i cijene na razvojnem hardveru — rasprava o tome je u Poglavlju 11.

10.3 Latencija

Generiranje okvira nužno podiže latenciju: pravi okvir t mora se prikazati istovremeno s (ili nakon) generiranim okvirom koji leži između $t - 1$ i t , pa ravnomjeran prikaz zahtijeva čekanje reda veličine pola render perioda. Izmjereno (Poglavlje 7), latencija do pravog okvira raste za +11 do +16 ms naspram prikaza bez generiranja, ovisno o opterećenju — približno jedan render period, kako i analiza mehanizma predviđa. Ovo je prikazano kao cijena tehnologije, ne sakriveno u prosjeku FPS-a.

10.4 Provjera na dva GPU-a proizvođača

Vidi odjeljak [10.1](#): kvaliteta se poklapa preko AMD/NVIDIA granice na 10^{-3} – 10^{-1} dB, a performanse po modulu pokazuju dosljedan arhitekturni obrazac (AMD brži na aritmetički-intenzivnim prolazima procjene gibanja i naučene mješavine, NVIDIA brži na teksturno-intenzivnom akumulacijskom prolazu upscalera), ne slučajan šum.

10.5 Artefakti

Vizualne usporedbe (uvećani izresci kroz sve upscalere, HSV prikaz procjene gibanja naspram egzaktnog polja, usporedba interpoliranog okvira i 50/50 blenda, usporedba komponiranog i upečenog korisničkog sučelja) generiraju zasebne skripte iz `scripts/` direktorija i nisu uključene ovdje kao zaseban skup slika, osim demonstracije naučene komponente (Slike [8.1–8.4](#)) koja je izravno relevantna za zaključak Poglavlja [8](#).

11 | Rasprava i ograničenja

11.1 Odstupanja od izvornog plana

Metrike kvalitete. Izvorni plan predviđao je PSNR, SSIM, FLIP i LPIPS. Implementirani su samo PSNR i SSIM, oba na GPU-u. FLIP i LPIPS zahtijevaju predtre-nirane perceptivne modele odnosno CIE-prostor razlike percepcije koji nadilaze GPU compute shadere ovog projekta; PSNR/SSIM su se u praksi pokazali dovoljno osjetljivi da razlikuju sve ablacije izvještene u poglavljima 5–8, uz poznatu manu (PSNR nagrađuje zamućenje na aliasiranom sadržaju, djelomično riješeno supersampliranom referencom — Poglavlje 9) koja je svugdje gdje je relevantna i eksplicitno navedena uz brojku.

Usporedba s Lucas–Kanade metodom. Plan je predviđao opcijski dodatak: implementaciju Lucas–Kanade gradijentne metode za procjenu gibanja i usporedbu točnosti/brzine s implementiranim block matchingom. Ovo nije rađeno u dostupnom vremenu; ostaje otvoreno kao smjer nastavka (§12.2).

Naučena komponenta se ne isplati na razvojnom hardveru. Poglavlje 8 mjeri, ne pretpostavlja, da mala naučena mreža na GPU-u bez matričnih/tenzorskih jedinica nosi cijenu (2,7 ms po okviru na 1080p) koja nije opravdana dobitkom kvalitete (0,10–0,20 dB). Ovo nije neuspjeh implementacije nego izravno mjerenje razloga zašto komercijalna rješenja (DLSS, FSR4/Redstone, PSSR) traže namjenski ML hardver — pitanje koje je opis teme postavio, s brojkom kao odgovorom.

11.2 Ograničenja mjerne postavke

Jedna primarna testna scena. Sponza je geometrijski bogata (tanka geometrija, stupovi, natpisi) ali svjetlosno jednostavna (jednostavan forward shading, mip-mapirane teksture s malo aliasinga). Posljedica je mjerljiva u poglavljima 5 i 7: ni upscaler ni generiranje okvira na ovoj sceni ne *ubrzavaju* render (cijena mehanizma je usporediva ili veća od cijene renderiranja lagane scene), pa se dobitak izvještava kao kvaliteta, ne kao FPS pri istoj sceni. Sekundarna proceduralna scena (pokretni objekti) koristi se posvuda gdje je gibanje objekata, a ne samo kamere, relevantno (ablacije generiranja okvira i naučene mješavine).

Aproksimacija latencije. Latencija se mjeri od uzorkovanja ulaza do *swapa* (predaje kompozitoru/monitoru), ne do stvarnog fotona na zaslonu — razlika koju bi trebao mjeriti vanjski senzor (fotodioda ili kamera visoke brzine), izvan opsega ovog rada. Usporedbe između načina prikaza u ovom radu (s generiranjem, bez njega, s pacingom, bez njega) su i dalje valjane jer sve mjere isti dodatak

na istom stogu.

Sub-pixel preciznost procjene gibanja. Implementirani optical flow (Poglavlje 6) radi cjelobrojnu pretragu na luminanciji pola prikazne razlučivosti — najmanji razlučiv pomak je dva prikazna piksela. Parabolička interpolacija oko minimuma kriterija podudaranja bi smanjila kvantizacijsku pogrešku i ostaje kao poznat, nenapravljen korak.

12 | Zaključak i mogući nastavak

12.1 Doprinosi

Ovaj rad je od nule, u C++20 i OpenGL 4.6 compute shaderima, implementirao kompletan cjevovod temporalnog upscalinga i generiranja interpoliranih okvira po uzoru na AMD FSR3: prostorni i temporalni upscaler s dilatacijom vektora, disokluzijom, lockovima i adaptivnom rekonstrukcijom (Poglavlje 5); piramidalnu procjenu gibanja iz slike (Poglavlje 6); generiranje međuokvira s frame pacingom, latencijom i kompozicijom korisničkog sučelja (Poglavlje 7); i malu naučenu komponentu koja uči birati između klasičnih kandidata (Poglavlje 8). Svaki dio je mjeran naspram vlastite *ground-truth* reference — pravog okvira i pravog međuokvira koje samo vlastiti renderer može proizvesti — pa su tvrdnje u ovom radu brojke, ne dojam.

Svaki korak cjevovoda mjerljivo nadmašuje prethodni (Tablica 10.3), i cijeli mjerni paket (283 konfiguracije) reproduciran je i provjeren na dva neovisna GPU-a dvaju različitih proizvođača (odjeljak 10.1), uz kvalitetu koja se poklapa preko te granice na 10^{-3} – 10^{-1} dB. Rad odgovara i na pitanje koje njegov opis izravno postavlja — isplati li se naučena komponenta bez namjenskog ML hardvera — mjerenjem, ne pretpostavkom: ne isplati se, i to je izmjeren, obrazložen odgovor (Poglavlje 8), ne izostanak rezultata.

12.2 Mogući nastavak

- **Veći naučeni model uz namjenski ML hardver.** Analiza cijene (Poglavlje 8) pokazuje da trend kvalitete s veličinom mreže nije zasićen (najveći testirani model je i najbolji), a granica je hardverska, ne algoritamska — isti model bi na GPU-u s matričnim/tenzorskim jedinicama nosio bitno manji dio cijene okvira.
- **Usporedba s Lucas–Kanade metodom procjene gibanja** (opsijski dodatak iz izvornog plana, §11) — gradijentna metoda naspram implementiranog block matchinga, po točnosti i cijeni.
- **FLIP i LPIPS** kao dodatne perceptivne metrike, uz postojeći PSNR/SSIM, posebno za slučajeve gdje se ta dva mjerenja razilaze (Poglavlje 5, aliasirani sadržaj).

- **Sub-pixel profinjenje procjene gibanja** (parabolička interpolacija oko minimuma kriterija podudaranja), koje bi smanjilo kvantizacijsku grešku opisanu u Poglavlju 11.
- **Provjera periodičnosti unutar block matchinga**, koja bi riješila preostali izvor greške na periodičnim teksturama identificiran u Poglavlju 7 (težina toka uz game vektor).

Literatura

- [1] D. FLEET, Y. WEISS, *Optical Flow Estimation*, u: *Handbook of Mathematical Models in Computer Vision*, Springer, 2006.
- [2] B. KARIS, *High Quality Temporal Supersampling*, ACM SIGGRAPH Courses: Advances in Real-Time Rendering, 2014.
- [3] L. YANG, S. NEHAB, P. V. SANDER, et al., *A Survey of Temporal Antialiasing Techniques*, Computer Graphics Forum **39**(2) (2020), 607–621.
- [4] Z. HUANG, T. ZHANG, W. HENG, B. SHI, S. ZHOU, *Real-Time Intermediate Flow Estimation for Video Frame Interpolation*, ECCV 2022. (arXiv:2011.06294)
- [5] F. REDA, J. KONTKANEN, E. TABELLION, et al., *FILM: Frame Interpolation for Large Motion*, ECCV 2022. (arXiv:2202.04901)
- [6] Z. WANG, A. C. BOVIK, H. R. SHEIKH, E. P. SIMONCELLI, *Image Quality Assessment: From Error Visibility to Structural Similarity*, IEEE Transactions on Image Processing **13**(4) (2004), 600–612.
- [7] AMD GPUOpen, *FidelityFX Super Resolution 3 — Frame interpolation, Optical flow, Super resolution temporal*. Dostupno na https://gpuopen.com/manuals/fidelityfx_sdk/. (dokumentacija korištena kao referenca, implementacija u ovom radu je vlastita — vidi docs/reference/README.md u repozitoriju rada)
- [8] NVIDIA, *DLSS 2.0: A Big Leap In AI Rendering i DLSS 3: AI-Powered Neural Graphics Innovations*. Dostupno na <https://www.nvidia.com/en-us/geforce/news/>.
- [9] Intel, *XeSS 2 Whitepaper*. Dostupno na <https://www.intel.com/content/www/us/en/develop>
- [10] Arm, *How Arm Neural Super Sampling Works*. Dostupno na <https://developer.arm.com/community/arm-community-blogs/>.
- [11] NVIDIA, *Reflex 2: Even Lower Latency Gameplay With Frame Warp*. Dostupno na <https://www.nvidia.com/en-us/geforce/news/>.

Sažetak

Razlučivost i frekvencija osvježavanja zaslona rastu brže od rasterizacijske snage GPU-a, pa igre sve češće rekonstruiraju sliku iz manjeg broja stvarno izračunatih uzoraka — kroz prostor (upscaling) i kroz vrijeme (generiranje interpoliranih okvira). Ovaj rad iz opisa tehnika AMD FidelityFX Super Resolution 3 (FSR3), a ne iz izvornog koda, iznova implementira cijeli klasičan cjevovod u C++20 i OpenGL 4.6 compute shaderima: temporalni upscaler s dilatacijom vektora gibanja, odbacivanjem povijesti po dubini, zaštitom tankih detalja (lockovi) i adaptivnom rekonstrukcijom; piramidalnu procjenu gibanja iz slike (optical flow) za sadržaj koji igrini vektori ne opisuju; i generiranje međuokvira s frame pacingom, latencijom i kompozicijom korisničkog sučelja. Uz to je implementirana i mala naučena komponenta koja uči birati između klasičnih kandidata mješavine umjesto ručno napisane heuristike.

Ključna postavka je vlastiti renderer testne scene: kako se scena i kamera u cijelosti kontroliraju, moguće je renderirati i *ground truth* — puni native okvir i pravi među-okvir na trenutku $t - 0,5\Delta t$ — čime kvaliteta postaje PSNR/SSIM brojka naspram poznate istine, a ne subjektivan dojam. Svaki dio cjevovoda mjerljivo nadmašuje prethodni: puni upscaler dobiva do +4,56 dB nad prostornim baseline-om, procjena gibanja uklanja do 87% gibanja koje igrini vektori ne vide, a generiranje okvira dobiva 7–10 dB nad naivnim spajanjem dva okvira. Naučena komponenta mjerljivo podiže kvalitetu (SSIM na svih osam mjerenih pogleda), ali se na GPU-u bez matričnih/tenzorskih jedinica ne isplati — cijena joj nije opravdana dobitkom, što je izmjeren, a ne pretpostavljen odgovor na to koliko daleko ide analitički pristup bez namjenskog ML hardvera. Cijeli mjerni paket od 283 konfiguracije reproduciran je i provjeren na dva GPU-a različitih proizvođača, s kvalitetom koja se poklapa preko te granice.

Ključne riječi

upscaling, generiranje interpoliranih okvira, temporalni antialiasing, procjena gibanja, optical flow, GPU compute shaderi, OpenGL, FSR3, naučena mješavina

A UI System for Dynamic Resolution Upscaling and Artificial Frame Generation in Interactive Graphics

Summary

Display resolution and refresh rate grow faster than GPU rasterisation throughput, so games increasingly reconstruct the image from fewer actually rendered samples, both spatially (upscaling) and temporally (interpolated frame generation). This thesis reimplements, from the published description of AMD FidelityFX Super Resolution 3 (FSR3) rather than from its source code, the full classical pipeline in C++20 and OpenGL 4.6 compute shaders: a temporal upscaler with motion-vector dilation, depth-based history rejection, thin-detail protection (locks) and adaptive reconstruction; a pyramidal image-space motion estimator (optical flow) for content that engine motion vectors do not describe; and interpolated-frame generation with frame pacing, latency, and UI compositing. A small learned component is also implemented, which learns to pick among the classical blend candidates instead of a hand-written heuristic.

The central design choice is a custom test-scene renderer: because the scene and camera are fully controlled, it can render *ground truth* — a full native frame and a true interpolated frame at $t - 0.5\Delta t$ — so quality becomes a PSNR/SSIM number against a known answer, not a subjective impression. Every pipeline stage measurably improves on the previous one: the full upscaler gains up to +4.56 dB over the spatial baseline, motion estimation recovers up to 87% of motion that engine vectors miss, and frame generation gains 7–10 dB over naively blending two frames. The learned component measurably improves quality (higher SSIM on all eight measured views) but does not pay for itself on a GPU without matrix/tensor units — an answer obtained by measurement, not assumption, to how far an analytical approach can go without dedicated ML hardware. The full measurement suite (283 configurations) was reproduced and cross-checked on two GPUs from different vendors, with quality metrics agreeing across that boundary.

Keywords

upscaling, interpolated frame generation, temporal antialiasing, motion estimation, optical flow, GPU compute shaders, OpenGL, FSR3, learned blending

Životopis

Ante Galić

[PLACEHOLDER: životopis — Ante Galić, prvo lice jednine]

Ivo Janković

[PLACEHOLDER: životopis — Ivo Janković, prvo lice jednine]